

With at least two examples, create a file and explain in the simplest way all operators, datatypes, conditions and loops.

LOOPS

Loops are a flow control structure that repeats a block of code instead of writing it over again for as long as the condition is true but stops as soon as the condition becomes false.

For loops

The for loop is used when we know the number of times a block of code will be repeated

Take

```
names = ["joe", "lin", "tila", "jet", "mei"]
for item in names:
    print(item)

grasses = ["rye", "wheat", "barley", "sugarcane", "corn"]
for grass in grasses:
    print(grass)

numbers = {1,2,3,4,5,6}
#would print values, 2,3,4 only
for value in range(2,5):
    print(value)
```

while loops

these are used when you want to print a value until the condition changes. liken it to a restaurant, the manager tells the workers to keep serving as long as the customers keep coming. At that point, the only thing that would stop the continuity for service would be either the completion of the food or no more customers coming in.

let's say:

```
water_level = 0
while water_level < 10:
    print("levels are low")
    water_level += 2

num = 1
while num < 5:
    print("the while loop")
    num += 1
else:
    print("hehe still looping")
```

Nested loop

These are loops within loops

```
#nested loops
rooms = [["joy","pal","mei"], ["duff", "tay", "ray"], ["fei", "wer", "don"]]
for room in rooms:
    for item in rooms:
        print(item)
```

CONDITIONS (if, if...else, if...elif...elif...else)

The if condition: runs condition only if condition is true

```
# if
age = 20
if age >= 18:
    print("Adult")

names=["joe", "isa", "fei", "noe"]
#checking for the name loise
if name != "loise":
    print(names)
```

The if...else: chooses between two possibilities

```
# if else
age = 15
if age >= 18:
    print("Adult")
else:
    print("Minor")

numbers = [18,30,40,70]
#conditions with some alternatives / options
if 80 in numbers:
    print("80 is found")
else:
    print("80 is not found")
```

The if...elif...elif: this records multiple conditions

```
#if elif else
score = 75
```

```

if score >= 90:
    print("Grade A")
elif score >= 70:
    print("Grade B")
else:
    print("Grade C")

if age >= 15:
    print("you qualify for retirement fund")
elif age >= 18 and age < 50:
    print("you are working for your retirement")
elif age < 18:
    print("child labour")

```

DATA TYPES

They specify the type of information that can be stored in a variable.

Numeric (int, float, complex): holds numeric values

Integer (int): stores whole numbers that is to say without decimals

```

# int
num = 56
print(num)

t = -5
print(t)

# in case you chose to print(b), python would give you the exact value because it
knows it's a float
# but know that int consists of whole numbers only
a, b = 4, -5.5
print(a)

```

Float: stores values with decimal places

```

#float
# you need to know that for as long as it has a decimal place it becomes a float
val = 2.5
print(val)

c, d = 6, 12.4
#at the next step you have chosen to print out the float

```

```
print(d)
```

Complex: stores numbers with negative square roots (complex numbers). These numbers are in the form $(x + iy)$ with x being the real part (which is any number be it decimal or whole number without a negative root) and iy being the imaginary part (which is a number with a negative root). However, in python, the imaginary part is usually denoted by a "j" a complex number would be presented in the form $3 + 4j$

```
#complex
e = 4 + 9j
print(e)

# the output would be -5j because it is purely imaginary
g = 0 - 5j
print(g)

# the output would be 3j because, (3* (0+1j)) = 3j
h = 3 * 1j
print(h)
```

Strings (str): holds a sequence of characters. They are used to represent text such as words, sentences and characters. As long as something is in quotes whether double or single

```
# strings
name = "noel"
print(name)

age = "20"
print(age + "years")
```

Sequences (list, tuple, range): holds a collection of items. The information in sequences is accessed by indexing.

Lists: the items in lists can be changed by adding or removing that is to say they are mutable

```
# lists
students = ["paul", "zoe", "ker", "tade"]
# would print paul
print(students[0])

marks = [10, 20, 30, [40,50], [60], 70, [80,[90, 100]]]
# output is 10
print(marks[0])
# output is 50
```

```
print(marks[3][1])
# output is 100
print(marks[6][1][1])
#output is70
print([5])
```

Tuples: these are sequences in which no changes can be made that is to say the way we add and remove items to a list, it can not be done for tuples.

```
#tuples, be keen on the kinds of brackets
coordinates = (12, 15, 16, 83)
# output is 83
print(3)

names = ("joel","theo","ker")
print(names[0])
print(names[-1])
```

Range: creates a sequence of integers from the first value and stops one value before the suggested number.

```
r = range(4)
#values in the range of 4 are 0,1,2,3
print(r)

q = range(3, 7)
# range says, get the starting value aka the lower limit and list it then stop
before the upper limit
print(q)
```

Mappings(dict): data is held in key value pairs

Dictionary is an ordered collection of items. Information is stored in key/value pairs

```
#dictionary
#kenya becomes the key to nairobi meaning to access nairobi one would have to use
the key kenya
cities = {"uganda":"kampala","kenya":"nairobi", "france":"paris",
"egypt":"cairo", "mexico":"cdmx"}
# would print kampala
print(cities["uganda"])
# would print all the keys and in this case the countries
print(cities.keys())
print(cities.values())
```

```
student = {"name": "dave", "age": "30", "location": "mutungo"}
print(student["name"])
print(student.keys())
```

Boolean(bool): executes either “true” or “false”

This is basically the lie detector of python. The answer is always either true or false and never intermediate

```
#boolean
x = True
y = False
print(x)

print(5 > 3)
print(5 < 3)
print(10 == 10)

age = 20
if age >= 18:
    print("Adult")
```

Set (set, frozen): holds collection of unique items

Set: an unordered collection of unique items .a set removes duplicate values that is to say no repetition and the values are not printed in a certain order meaning that no indexing is allowed

```
#set
numbers = {1, 2, 3, 4}
print(numbers)

A = {1,2,3,4}
B = {3,4,5,6}
# difference meaning get what is in a but not in b simply put A'
print(A - B)
```

Frozen: is a set class that gets locked once it has been created that is to say it is more like a tuple whereby no additions or removals of items is done meaning it is immutable

```
#frozen
f = frozenset([1,2,3,4])
print(f)
```

OPERATORS

These are symbols, characters or anything in a statement that tells a computer what to do with values

Arithmetic operators: these perform everyday mathematical calculations such as multiplication, addition, subtraction and division

```
# arithmetic operators
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

Assignment operators: these are used to assign values to variables. They let python store a value or update an existing value. The most common one being “=” read as “assign to”

```
#assignment operators
score = 50

score += 10
print(score)

score -= 5
print(score)

score *= 2
print(score)

score /= 5
print(score)
```

Comparison operators: these compare two values and evaluate whether the condition is true or false that is to say they return a Boolean result such as; == (is equal to), != (not equal to), > (greater than), < (less than)

```
# comparison operators
x = 15
y = 10

print(x == y)
print(x != y)
```

```
print(x > y)
print(x < y)
print(x >= y)
print(x <= y)
```

Logical operators: these are used to combine multiple conditional states such they include “and”, “or”, “not”. “and”; both conditions must be fulfilled. “or”; as long as one condition is fulfilled. “not”; neither condition is fulfilled that is to say it turns a true into false and vice versa.

```
# logical operators
age = 20
student = True

print(age >= 18 and student)
print(age < 18 or student)
print(not student)
```

Membership operators: used to verify whether a value “in” or “not in” within a sequence

```
# membership operators
student = {"name": "John", "age": 20}
print("name" in student)
```

Identity operators: these check whether two variables point to the same object in memory and not whether they have the same value. Usually denoted by “is” or “is not”

```
# identity operators
a = [1,2,3]
b = a
# output is True
print(a is b)

a = [1,2,3]
b = [1,2,3]
# output is True
print(a is not b)

a == b
print()

a is b
print()
```



```
fruits = ["apple","orange","mango"]  
print("mango" in fruits)
```

Bitwise operators: these work on binary digits that is zeros and ones